

INVENTORY MANAGEMENT DASHBOARD

PROGRAM OUTPUT SCREENSHOT

The screenshot shows a web application window titled "Inventory Management Dashboard". The interface includes a menu bar with "File" and "Actions", and a main title "INVENTORY MANAGEMENT DASHBOARD". On the left, there is a "Product Details" section with input fields for "Product ID:", "Product Name:", "Price (\$):", and "Quantity:". Below these fields are four buttons: "Add Product", "Update Product", "Delete Product", and "Clear". On the right, there is a "Search:" input field with a "Filter" button. Below the search field is a table displaying inventory data.

ID	Name	Price (\$)	Quantity	Total Value (\$)
P101	Laptop Pro	1299.99	45	58499.55
P102	Wireless Mouse	29.99	150	4498.50
P103	Mechanical Keyb.	89.99	80	7199.20
P104	HD Monitor 27"	249.99	35	8749.65

SOURCE CODE

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import javax.swing.table.TableRowSorter;
import java.awt.*;
import java.util.ArrayList;
import java.util.List;

public class InventoryManagementDashboard extends JFrame {
    private final JTextField idField = new JTextField();
    private final JTextField nameField = new JTextField();
    private final JTextField priceField = new JTextField();
    private final JTextField qtyField = new JTextField();
    private final JTextField searchField = new JTextField();

    private final DefaultTableModel tableModel;
    private final JTable productTable;
    private final List<Product> productList = new ArrayList<>();
    private TableRowSorter<DefaultTableModel> rowSorter;

    public InventoryManagementDashboard() {
        setTitle("Inventory Management Dashboard");
        setSize(850, 550);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JPanel mainPanel = new JPanel(new BorderLayout(12, 12));
        mainPanel.setBorder(new EmptyBorder(15, 15, 15, 15));

        JLabel titleLabel = new JLabel("INVENTORY MANAGEMENT DASHBOARD",
SwingConstants.CENTER);
        titleLabel.setFont(new Font("SansSerif", Font.BOLD, 22));
        mainPanel.add(titleLabel, BorderLayout.NORTH);

        JPanel formPanel = new JPanel(new GridLayout(5, 2, 8, 8));
        formPanel.setBorder(BorderFactory.createTitledBorder("Product Details"));
        formPanel.setPreferredSize(new Dimension(300, 200));

        formPanel.add(new JLabel("Product ID:"));
        formPanel.add(idField);
        formPanel.add(new JLabel("Product Name:"));
        formPanel.add(nameField);
        formPanel.add(new JLabel("Price ($) :"));
        formPanel.add(priceField);
        formPanel.add(new JLabel("Quantity:"));
        formPanel.add(qtyField);

        JButton addBtn = new JButton("Add Product");
        JButton updateBtn = new JButton("Update Product");
        JButton deleteBtn = new JButton("Delete Product");
        JButton clearBtn = new JButton("Clear");

        JPanel buttonPanel = new JPanel(new GridLayout(2, 2, 6, 6));
        buttonPanel.add(addBtn);
        buttonPanel.add(updateBtn);
        buttonPanel.add(deleteBtn);
        buttonPanel.add(clearBtn);

        JPanel leftPanel = new JPanel(new BorderLayout(8, 8));
        leftPanel.add(formPanel, BorderLayout.CENTER);
        leftPanel.add(buttonPanel, BorderLayout.SOUTH);
    }
}
```

```

mainPanel.add(leftPanel, BorderLayout.WEST);

JPanel rightPanel = new JPanel(new BorderLayout(8, 8));

JPanel searchPanel = new JPanel(new BorderLayout(6, 6));
searchPanel.add(new JLabel("Search: "), BorderLayout.WEST);
searchPanel.add(searchField, BorderLayout.CENTER);
JButton searchBtn = new JButton("Filter");
searchPanel.add(searchBtn, BorderLayout.EAST);
rightPanel.add(searchPanel, BorderLayout.NORTH);

tableModel = new DefaultTableModel(new String[]{"ID", "Name", "Price ($)",
"Quantity", "Total Value ($)"}, 0) {
    @Override
    public boolean isCellEditable(int row, int column) {
        return false;
    }
};
productTable = new JTable(tableModel);
rowSorter = new TableRowSorter<>(tableModel);
productTable.setRowSorter(rowSorter);
productTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

JScrollPane tableScroll = new JScrollPane(productTable);
rightPanel.add(tableScroll, BorderLayout.CENTER);
mainPanel.add(rightPanel, BorderLayout.CENTER);

createMenuBar();

addBtn.addActionListener(e -> addProduct());
updateBtn.addActionListener(e -> updateProduct());
deleteBtn.addActionListener(e -> deleteProduct());
clearBtn.addActionListener(e -> clearFields());
searchBtn.addActionListener(e -> filterTable());

searchField.addCaretListener(e -> filterTable());

productTable.getSelectionModel().addListSelectionListener(e -> {
    int selectedRow = productTable.getSelectedRow();
    if (selectedRow >= 0) {
        int modelRow = productTable.convertRowIndexToModel(selectedRow);
        idField.setText(tableModel.getValueAt(modelRow, 0).toString());
        nameField.setText(tableModel.getValueAt(modelRow, 1).toString());
        priceField.setText(tableModel.getValueAt(modelRow, 2).toString());
        qtyField.setText(tableModel.getValueAt(modelRow, 3).toString());
        idField.setEditable(false);
    }
});

add(mainPanel);
setVisible(true);

loadSampleData();
}

private void createMenuBar() {
    JMenuBar menuBar = new JMenuBar();
    JMenu fileMenu = new JMenu("File");
    JMenuItem exitItem = new JMenuItem("Exit");
    exitItem.addActionListener(e -> System.exit(0));
    fileMenu.add(exitItem);

    JMenu actionMenu = new JMenu("Actions");
    JMenuItem refreshItem = new JMenuItem("Reset Filter");

```

```

        refreshItem.addActionListener(e -> {
            searchField.setText("");
            filterTable();
        });
        actionMenu.add(refreshItem);

        menuBar.add(fileMenu);
        menuBar.add(actionMenu);
        setJMenuBar(menuBar);
    }

    private void loadSampleData() {
        productList.add(new Product("P101", "Laptop Pro", 1299.99, 45));
        productList.add(new Product("P102", "Wireless Mouse", 29.99, 150));
        productList.add(new Product("P103", "Mechanical Keyboard", 89.99, 80));
        productList.add(new Product("P104", "HD Monitor 27\"", 249.99, 35));
        refreshTable();
    }

    private void refreshTable() {
        tableModel.setRowCount(0);
        for (Product p : productList) {
            tableModel.addRow(new Object[]{
                p.id, p.name, String.format("%.2f", p.price), p.quantity,
                String.format("%.2f", p.getTotalValue())
            });
        }
    }

    private void addProduct() {
        if (!validateInput()) return;
        String id = idField.getText().trim();

        for (Product p : productList) {
            if (p.id.equalsIgnoreCase(id)) {
                JOptionPane.showMessageDialog(this, "Product ID already exists.", "Error",
                    JOptionPane.ERROR_MESSAGE);
                return;
            }
        }

        Product p = new Product(
            id,
            nameField.getText().trim(),
            Double.parseDouble(priceField.getText().trim()),
            Integer.parseInt(qtyField.getText().trim())
        );
        productList.add(p);
        refreshTable();
        JOptionPane.showMessageDialog(this, "Product added successfully.", "Success",
            JOptionPane.INFORMATION_MESSAGE);
        clearFields();
    }

    private void updateProduct() {
        int selectedRow = productTable.getSelectedRow();
        if (selectedRow < 0) {
            JOptionPane.showMessageDialog(this, "Select a product from the table to
update.", "Warning", JOptionPane.WARNING_MESSAGE);
            return;
        }
        if (!validateInput()) return;

        int modelRow = productTable.convertRowIndexToModel(selectedRow);
    }

```

```

String id = (String) tableModel.getValueAt(modelRow, 0);

for (Product p : productList) {
    if (p.id.equals(id)) {
        p.name = nameField.getText().trim();
        p.price = Double.parseDouble(priceField.getText().trim());
        p.quantity = Integer.parseInt(qtyField.getText().trim());
        break;
    }
}
refreshTable();
JOptionPane.showMessageDialog(this, "Product updated successfully.", "Success",
JOptionPane.INFORMATION_MESSAGE);
clearFields();
}

private void deleteProduct() {
    int selectedRow = productTable.getSelectedRow();
    if (selectedRow < 0) {
        JOptionPane.showMessageDialog(this, "Select a product to delete.", "Warning",
JOptionPane.WARNING_MESSAGE);
        return;
    }
    int modelRow = productTable.convertRowIndexToModel(selectedRow);
    String id = (String) tableModel.getValueAt(modelRow, 0);

    int choice = JOptionPane.showConfirmDialog(this, "Are you sure you want to delete
this product?", "Confirm Deletion", JOptionPane.YES_NO_OPTION);
    if (choice == JOptionPane.YES_OPTION) {
        productList.removeIf(p -> p.id.equals(id));
        refreshTable();
        JOptionPane.showMessageDialog(this, "Product deleted successfully.", "Success",
JOptionPane.INFORMATION_MESSAGE);
        clearFields();
    }
}

private void filterTable() {
    String query = searchField.getText().trim();
    if (query.isEmpty()) {
        rowSorter.setRowFilter(null);
    } else {
        rowSorter.setRowFilter(RowFilter.regexFilter("(?i)" + query));
    }
}

private void clearFields() {
    idField.setText("");
    nameField.setText("");
    priceField.setText("");
    qtyField.setText("");
    idField.setEditable(true);
    productTable.clearSelection();
}

private boolean validateInput() {
    if (idField.getText().trim().isEmpty() ||
        nameField.getText().trim().isEmpty() ||
        priceField.getText().trim().isEmpty() ||
        qtyField.getText().trim().isEmpty()) {
        JOptionPane.showMessageDialog(this, "All product fields are required.",
"Validation Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }
}

```

```

        try {
            double price = Double.parseDouble(priceField.getText().trim());
            if (price < 0) {
                JOptionPane.showMessageDialog(this, "Price must be non-negative.",
"Validation Error", JOptionPane.ERROR_MESSAGE);
                return false;
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Price must be a valid number.",
"Validation Error", JOptionPane.ERROR_MESSAGE);
            return false;
        }
        try {
            int qty = Integer.parseInt(qtyField.getText().trim());
            if (qty < 0) {
                JOptionPane.showMessageDialog(this, "Quantity must be non-negative.",
"Validation Error", JOptionPane.ERROR_MESSAGE);
                return false;
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Quantity must be an integer.", "Validation
Error", JOptionPane.ERROR_MESSAGE);
            return false;
        }
        return true;
    }

    static class Product {
        String id, name;
        double price;
        int quantity;

        Product(String id, String name, double price, int quantity) {
            this.id = id;
            this.name = name;
            this.price = price;
            this.quantity = quantity;
        }

        double getTotalValue() {
            return price * quantity;
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(InventoryManagementDashboard::new);
    }
}

```